Student notes — Session 9 · Linear arrays

Slides: [`Class_09_Linear_arrays.pdf`](../slides/Class_09_Linear_arrays.pdf). These notes are the notes pane of the student copy of the deck.

Slide 1 — Linear arrays

The programs of this class are on Blackboard: Week 5 module, *Class 9 code*. It is a zip file. Unzip it and open the folder in Visual Studio Code. Every file has a comment at the top: what the program does, how to run it, and a list called **Try** with small changes to make. Make them.

How to run a program. Same as every week. In the terminal type `gcc -Wall name.c -o name` and press Enter, then `./name` (or `.\name` in Windows PowerShell) and press Enter.

Until now one variable held one value. Today one name holds many values: an **array**.

Slide 2 — Quiz 2 is back

Your mark is on your paper and in Blackboard. Open the course, click **Gradebook**, then **Quiz 2**. You see your mark, every comment from your paper, and your marked paper as a PDF file.

Three ideas cost most of the marks in the class. Slides 3 to 11 show them, one at a time. All three come back in today's array loops.

Slide 3 — Mistake 1: trace the loop, round by round

Do not guess what a loop prints. Make a table with one row per round: the value of `x`, the test, what is printed, and `count`.

This is Question 1(a) of variant A:

```
int x = 1;
int count = 0;
while (x < 20) {
    printf("%d ", x);
    if (x % 3 == 0) {
        count++;
    }
    x = x * 2 + 1;
}
printf("\nx = %d, count = %d\n", x, count);
```

`x`	`x < 20`?	prints	`count`
1	yes	`1`	0
3	yes	`3`	1
7	yes	`7`	1
15	yes	`15`	2
31	no	nothing	2

Every `x` that passes the test is printed: `1 3 7 15`.

Slide 4 — The loop stops on the first no

After 15, `x = x * 2 + 1` makes `x` 31. The test `31 < 20` is false, so the loop stops. 31 is **never printed**. But `x` still holds 31: the loop does not undo its last step.

Slide 5 — So the two lines are

The program prints exactly this:

```
1 3 7 15
x = 31, count = 2
```

Two mistakes were common:

- Writing `x = 15`. After the loop, `x` holds the value that **failed** the test, 31.
- Writing only `2`. `printf` prints the words in its format too: `x = `, the comma, `count = `. Write the whole line.

Variant B was the same loop with `x = x * 2 + 2`, the test `x < 50`, and multiples of 5. It prints `2 6 14 30`, then `x = 62, count = 1`.

Slide 6 — Mistake 2: the counting program

Question 2 of variant A: read how many numbers the user will type, read them one by one, and print how many are even. Here is one complete answer:

```
#include <stdio.h>

int main(void) {
    int n, value;
    int count = 0;
    printf("How many numbers? ");
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) {
        printf("Number: ");
        scanf("%d", &value);
        if (value % 2 == 0) {
            count++;
        }
    }
    printf("Even numbers: %d\n", count);
    return 0;
}
```

When you type `5`, then `4 7 10 3 8`, the screen shows:

```
How many numbers? 5
Number: 4
Number: 7
Number: 10
Number: 3
Number: 8
Even numbers: 3
```

Variant B counted the scores of 50 or more. It is the same program with `if (score >= 50)`.

Slide 7 — Four places, every time

Every counting program has the same four places:

1. **Before the loop:** start the counter at 0: `int count = 0;`. Without `= 0` the counter starts with an unknown number.
2. **The loop:** it runs exactly `n` times: `for (int i = 1; i <= n; i++)`. Do not write `i <= 5`: the user chooses `n`.
3. **Inside the loop:** read one number, test it, and `count++;` when the test is true.
4. **After the loop:** print the result **once**.

Slide 8 — One } in the wrong place

Here the last `printf` is above the `}` that closes the loop, so it is **inside** the loop:

```
for (int i = 1; i <= n; i++) {
    printf("Number: ");
    scanf("%d", &value);
    if (value % 2 == 0) {
        count++;
    }
    printf("Even numbers: %d\n", count);
}
```

C does not look at the indent. It follows the braces. Move the `printf` below the `}` of the loop.

Slide 9 — Run it

With the `printf` inside the loop, and `3`, then `4 7 10` typed, the screen shows:

```
How many numbers? 3
Number: 4
Even numbers: 1
Number: 7
Even numbers: 1
Number: 10
Even numbers: 2
```

It prints a line after every number. The question wanted one line, at the end.

Slide 10 — Mistake 3: the compiler finds the small things

This code has four small slips:

```
int n, value;
int count = 0;
scanf("%d", &n);
for (i = 1; i =< n; i++) {
    scanf("%d"; &value);
    if (value % 2 == 0) {
        count++
    }
}
```

- `i` is never declared. Write `for (int i = 1; ...`.
- `=<` is not a C operator. Write `<=`.

- `scanf` needs a comma between the format and `&value`, not `;`.
- `count++` needs a `;` after it.

`gcc -Wall` stops at each one and names the line. These are its messages (shortened):

```
error: 'i' undeclared
error: expected expression before '<' token
error: expected ')' before ';' token
error: expected ';' before '}' token
```

Read the **first** error, fix it, and compile again. Often the next errors go away with it.

Slide 11 — All three come back today

Every array loop today runs exactly N times. You trace `i` and the box row by row, as on slide 3. You count with the four places of slide 7.

At home: type Question 2 into VS Code, compile it with `gcc -Wall`, run it with the example input, and compare it with your paper.

Slide 12 — What Friday gave us

What you need from Friday:

- A variable lives inside its `{ }`: that is its **scope**.
- A function gets a **copy** of each value.
- A new value goes back with `return`.
- Recursion: a **base case** and a **step closer**.

Slide 13 — Recap: what is on the screen?

Try to answer before you look at the next slide.

Slide 14 — Recap: what is on the screen?

The screen shows `5`. `add_ten` gets a **copy** of `x` in its own box `n`. It adds 10 to the copy. `x` in `main` stays 5. The file is `01_recap_add_ten.c`.

Slide 15 — Today's roadmap

Five parts:

1. What an array is: many boxes, one name.
2. A loop that visits every box.
3. Read values into an array, add them up, print them backwards.
4. The mistakes C does **not** stop.
5. Three small tasks for you.

Slide 16 — What an array is

Part 1: many values under one name.

Slide 17 — Five scores, five variables

Five quiz scores in five variables, `s0` to `s4`. The program prints them and their average:

```
7 9 6 10 8
average: 8.0
```

It works, but every new score needs a new name, and the `printf` line and the sum line both change. The file is `02_five_variables.c`.

Slide 18 — Now 35 students

With 35 students you would need 35 variables, 35 names in the `printf` and 35 names in the sum. A loop cannot help: a loop can change a number, but it cannot change a name.

Slide 19 — Five scores, one array

The same program with an **array**:

```
int scores[5] = {7, 9, 6, 10, 8};
```

One name, `scores`, with room for 5 `int` values. The values in the braces go into the boxes in order. The output is the same as before. The file is `03_five_array.c`.

Slide 20 — A loop walks through it

The loop makes `i` go 0, 1, 2, 3, 4. `scores[i]` is the value in box number `i`. So one `printf` line and one `sum` line reach all five boxes. For 35 students you change the 5; the loop stays the same.

Slide 21 — An array is a row of boxes

An array is a row of boxes:

- all boxes have the **same type**;
- the row has **one name**;
- each box has a number, its **index**.

You reach a box with the name and the index in square brackets: `scores[2]`. Think of lockers in a corridor: one row, a number on each door.

Slide 22 — int scores[5] = {7, 9, 6, 10, 8};

index	0	1	2	3	4
value	7	9	6	10	8

Five boxes. The index goes from **0** to **4**, not 1 to 5.

Slide 23 — Read the boxes

Try to answer before you look at the next slide. Use the table on slide 22.

Slide 24 — Read the boxes

- `scores[3]` is **10**. Count from 0: box 0 is 7, box 1 is 9, box 2 is 6, box 3 is 10.
- `scores[0] + scores[4]` is 7 + 8 = **15**.

If you said 6 for `scores[3]`, you counted from 1. Everybody does at first.

Slide 25 — The first index is 0

An array of 5 has the indexes 0, 1, 2, 3, 4.

- The first box: `scores[0]`.
- The last box: `scores[4]`, **not** `scores[5]`.
- An array of N boxes: indexes 0 to N-1.

Slide 26 — Make an array

```
int a[5];
```

- `int`: the type of every box.
- `a`: the name.
- `[5]`: how many boxes.

This array has no values yet. The file is `04_declare.c`.

Slide 27 — Put a value into one box

`a[0] = 10;` puts 10 into box 0. It works like `x = 10;`.

```
a[0] = 10;
a[1] = 20;
a[4] = 50;
```

`a[2]` and `a[3]` got no value. Do not print them: they hold unknown numbers, whatever was in memory before.

Slide 28 — Use one box

```
first: 10
second: 20
last: 50
```

`a[4]` is a value like any `int` variable: you can print it, add it, compare it.

Slide 29 — One box = one variable

Everything you did with `x`, you can do with `a[1]`:

- store: `a[1] = 20;`
- read: `printf("%d", a[1]);`
- change: `a[1] = a[1] + 5;` or `a[1]++;`

Slide 30 — Which index is February?

Try to answer before you look at the next slide. January is the first month.

Slide 31 — Which index is February?

February is month 2, so index **1**. December is month 12, so index **11**. The program prints:

```
Feb: 28
Dec: 31
```

Month number = index + 1. To get the days of a month the user typed, write `days[month - 1]`. The file is `05_days_in_month.c`.

Slide 32 — A short list

Try to answer before you look at the next slide. `a` has 5 boxes but only 2 values in its list.

Slide 33 — A short list

```
1 2 0 0 0
0 0 0 0 0
```

When you give a list in braces, the boxes with no value in the list get **0**. So `int b[5] = {0};` makes every box 0.

Careful: this works only with a list. `int a[5];` with no list does **not** give zeros. The file is `06_partial_list.c`.

Slide 34 — Any type works

`double prices[3] = {2.50, 1.20, 4.00};` is an array of three `double` values. The program changes `prices[1]` and prints all three:

```
item 0: 2.50 AZN
item 1: 1.35 AZN
item 2: 4.00 AZN
```

The file is `07_prices.c`.

Slide 35 — Making an array

- `int a[5];` five boxes, no values yet.
- `int a[5] = {1, 2, 3, 4, 5};` five boxes, five values.
- `int a[5] = {0};` five boxes, all 0.
- The size is fixed: the array cannot grow later.
- The indexes go from 0 to N-1.

Slide 36 — A loop over every box

Part 2: the loop you will write in almost every program with an array.

Slide 37 — The index can be a variable

The index can be a variable. `a[i]` means the box whose number is in `i`:

- when `i` is 0, `a[i]` is `a[0]`;
- when `i` is 3, `a[i]` is `a[3]`.

A `for` loop makes `i` go 0, 1, 2, ..., N-1. So one line inside the loop reaches every box.

Slide 38 — Print every box

```
for (int i = 0; i < 5; i++) {
    printf("a[%d] = %d\n", i, a[i]);
```

```
}
```

prints:

```
a[0] = 12
a[1] = 7
a[2] = 30
a[3] = 5
a[4] = 18
```

The first `%d` is the index `i`. The second is the value `a[i]`. The file is `08_index_and_value.c`.

Slide 39 — Step by step

The first time round, `i` is 0, so `a[i]` is `a[0]`, which is 12. The program prints `a[0] = 12`.

Slide 40 — Step by step

Then `i` is 1 (`a[1]` is 7), then 2 (`a[2]` is 30). The same line runs every time; only `i` changes, so it reaches a new box.

Slide 41 — Step by step

`i`	`a[i]`	**prints**
0	12	`a[0] = 12`
1	7	`a[1] = 7`
2	30	`a[2] = 30`
3	5	`a[3] = 5`
4	18	`a[4] = 18`
5		stop: `5 < 5` is false

Five boxes, five lines, no box missed.

Slide 42 — The array loop

```
for (int i = 0; i < 5; i++)
```

- It starts at **0**: the first box.
- It stops **before** 5: the last box is 4.
- For an array of N boxes, write `i < N`.

Slide 43 — Fill with a loop

A loop can also put values **into** an array:

```
for (int i = 0; i < 10; i++) {
    sq[i] = i * i;
}
```

Box 0 gets 0, box 1 gets 1, box 2 gets 4, box 3 gets 9, and so on. The file is `09_fill_squares.c`.

Slide 44 — Then print with a loop

A second loop prints the array:

```
0 1 4 9 16 25 36 49 64 81
```

One loop fills, one loop prints. You will see this often.

Slide 45 — Fill it with 1 to 10

Try to answer before you look at the next slide.

Slide 46 — Fill it with 1 to 10

```
sq[i] = i + 1;
```

`i` goes from 0 to 9, so `i + 1` goes from 1 to 10. With `sq[i] = i;` you would get 0 to 9.

Slide 47 — Read, add up, print backwards

Part 3: values from the keyboard.

Slide 48 — Read into the array

```
for (int i = 0; i < 5; i++) {
    scanf("%d", &a[i]);
}
```

`&a[i]` means: put the number **into** box `i`. The `&` is needed, like in `scanf("%d", &x);`. The file is `10_read_five.c`.

Slide 49 — Then print it back

If you type `4 8 15 16 23` and press Enter, the program prints:

```
Enter 5 numbers: 4 8 15 16 23
a[0] = 4
a[1] = 8
a[2] = 15
a[3] = 16
a[4] = 23
```

You can also type the numbers on five lines, pressing Enter after each one. `scanf` accepts both.

Slide 50 — Add them up

```
int sum = 0;
for (int i = 0; i < 5; i++) {
    sum += scores[i];
}
```

The total starts at 0, **before** the loop. With the input `7 9 6 10 8` the program prints:

```
sum: 40
average: 8.0
```

This is the running total from class 6, with `scores[i]` in place of `i`. The file is `11_sum_average.c`.

Slide 51 — The running total

`sum` starts at 0. When `i` is 0, `scores[0]` is 7, so `sum` becomes 7.

Slide 52 — The running total

`i`	`scores[i]`	`sum`
start		0
0	7	7
1	9	16
2	6	22
3	10	32
4	8	40

After the loop `sum` is 40. The average is `40 / 5.0`, which is 8.0. Divide by `5.0`, not `5`: `41 / 5` would give 8, but `41 / 5.0` gives 8.2.

Slide 53 — Print it backwards

Try to answer before you look at the next slide. The input is `1 2 3 4 5`, and you want `5 4 3 2 1`.

Slide 54 — Print it backwards

```
for (int i = 4; i >= 0; i--) {
    printf(" %d", a[i]);
}
```

- Start at the **last** index: 4.
- Go down: `i--`.
- Keep going while `i >= 0`.

The output is `Backwards: 5 4 3 2 1`. The file is `12_reverse.c`.

Slide 55 — Going down, step by step

`i` starts at 4, so the first number printed is `a[4]`, which is 5. Then `i` is 3 and it prints 4.

Slide 56 — Going down, step by step

`i`	`i >= 0`?	prints
4	true	5
3	true	4
2	true	3
1	true	2
0	true	1
-1	false	stop

Box 0 is printed because the test is `>=`. With `i > 0` the loop would stop at 1, and the first number would be missing.

Slide 57 — The size in one place

In the programs so far, the number 5 is written in three places. If you change one and forget another, you get a bug. Write the size once, at the top:

```
#define N 5
```

Everywhere below that line, gcc reads `N` as 5. There is no `=` and no `;` on this line. The name is in capital letters: everyone does it that way.

Slide 58 — #define N 5

```
#define N 5

int scores[N];
for (int i = 0; i < N; i++) {
```

To change the size, change one line. With the input `7 9 6 10 8` the program prints `average: 8.0`. The average is `(double) sum / N`: `sum` and `N` are both whole numbers, so `(double)` makes the division a real one. The file is `13_define_n.c`.

Slide 59 — Why keep every value?

Think about it before you look at the next slide: can you count the scores above the average without an array?

Slide 60 — Above the average

No. You know the average only **after** the last score. Then you need every score **again**, to compare it with the average. The array kept them all.

With the input `7 9 6 10 8`:

```
average: 8.0
above average: 2
```

The two scores above 8.0 are 9 and 10. The file is `14_above_average.c`.

Slide 61 — The mistakes C does not stop

Part 4: two mistakes that C does not stop for you, and one it does.

Slide 62 — Spot the bug

Try to answer before you look at the next slide. The array has five values. Look at the loop's test.

Slide 63 — One box too far

```
for (int i = 0; i <= 5; i++) {
    printf("%d\n", a[i]);
}
```

`i <= 5` makes `i` go up to 5, so the loop also reads `a[5]`. There is no `a[5]`: the last box is `a[4]`. The program still compiles with no warning, and it prints six lines: 10, 20, 30, 40, 50, and then a sixth number that is **not** in the array (a different number each time you run it: 88, then 4, then 234). The file is `15_bug_1e.c`.

The fix: `i < 5`.

Slide 64 — gcc -Wall says nothing

```
printf("last: %d\n", a[5]);
```

This compiles with no warning, runs, and prints a number that is not in the array, a different one on every run (`last: 136`, then `last: 183`). A wrong answer and no error message: this is why the mistake is dangerous. The file is `16_bug_index_5.c`. The fix: the last box is `a[4]`.

Slide 65 — gcc -O2 catches this one

If you compile with `gcc -Wall -O2 16_bug_index_5.c -o bug_index_5` (a capital letter O and the number 2), gcc looks harder and warns:

```
warning: array subscript 5 is above array bounds of 'int[5]'
```

But it does not catch every case. Never count on the compiler to find this bug.

Slide 66 — C does not check the index

C does not check that an index is between 0 and N-1.

- Reading outside the array: you get some number, maybe 0, maybe not.
- Writing outside the array: you may change **another** variable, or the program may crash.

So check it yourself: every array loop starts at `i = 0` and tests `i < N`.

Slide 67 — Copy an array?

Try to answer before you look at the next slide. Does `b = a;` compile?

Slide 68 — b = a does not compile

No. gcc says:

```
error: assignment to expression with array type
```

You cannot copy a whole array with `=`. The file is `17_bug_assign.c`.

Slide 69 — Copy box by box

Copy one box at a time, in a loop:

```
for (int i = 0; i < 3; i++) {
    b[i] = a[i];
}
```

Then the program changes `b[0]` to 100 and prints:

```
a: 1 2 3
b: 100 2 3
```

`b` is a real copy: changing it does not change `a`. The file is `18_copy_loop.c`.

Slide 70 — Three mistakes

mistake	what happens	fix
`i <= 5` for 5 boxes	reads `a[5]`: a wrong number	`i < 5`
`a[5]` as the last box	a wrong number, no error	`a[4]`
`b = a;`	does not compile	a loop: `b[i] = a[i];`

Slide 71 — Your turn

Part 5: three small programs. Write each one in VS Code, compile it and run it before you look at the answer.

Slide 72 — Task 1: a week of temperatures

Read the temperatures of the **7** days of a week into a `double` array. Print them in one line, then print their average. Test it with `18 21 19 23 25 22 20`.

Hint: `#define DAYS 7`, `double t[DAYS];`, and `%lf` in the `scanf`.

Slide 73 — Task 1: one answer

The file `19_task_temps.c`. It asks `Day 1:` to `Day 7:` (it prints `i + 1`, so the days are 1 to 7 while the index goes 0 to 6), then prints:

```
18.0 21.0 19.0 23.0 25.0 22.0 20.0
average: 21.1
```

The sum is 148, and 148 / 7 is 21.14. A `double` is read with `%lf`, not `%d` or `%f`.

Slide 74 — Task 2: double every value

Start from `int a[6] = {3, 8, 1, 5, 9, 2};`. Double every value **in** the array (change the boxes, not just what you print), then print the array.

Slide 75 — Task 2: one answer

```
for (int i = 0; i < 6; i++) {
    a[i] = a[i] * 2;
}
```

`a[i] *= 2;` does the same. A second loop prints `6 16 2 10 18 4`. The file is `20_task_double_all.c`.

Slide 76 — Task 3: how many are even?

Start from `int a[8] = {4, 7, 10, 3, 6, 6, 1, 12};`. Count how many values are even. Hint: the counter inside an `if` from class 6.

Slide 77 — Task 3: one answer

```
int even = 0;
for (int i = 0; i < 8; i++) {
    if (a[i] % 2 == 0) {
        even++;
    }
}
```

It prints `even values: 5`: the values 4, 10, 6, 6 and 12. The file is `21_task_even_count.c`.

Slide 78 — Today in one slide

- `int a[5];` five boxes, one name, one type.
- The index goes from **0** to **N-1**: `a[0]` to `a[4]`.
- The loop: `for (int i = 0; i < N; i++)`.
- `scanf("%d", &a[i]);` reads into a box.
- C does not check the index. `b = a;` does not compile.

Slide 79 — Before Friday

- **Knowledge check 9** on Blackboard covers this class. Three attempts, the best one is kept, and it does not count toward your grade.
- **Project 1** (Blackjack) is due **Sunday 11 October, 23:59**, on Blackboard. Upload early.
- **Quiz 3 is on Friday 16 October**, the first 25 minutes of the class, on paper. It covers classes 7 to 10: functions, scope, call-by-value, recursion, arrays, and Friday's class.
- Today's code is on Blackboard, Week 5 module, *Class 9 code*. Make the "Try" changes in each file.

Next class: **search an array**, find the smallest and the largest value, and **sort** it.